

CIS 301: Logical Foundations of Programming

Practice Exam 3 (Solution)

1. Use natural deduction to prove the following sequent:

```
(  
   $\exists (x: T) \Rightarrow (D(x) \wedge \neg B(x))$  ,  
   $\forall (x: T) \Rightarrow (\neg B(x) \rightarrow S(x))$   
)  
⊢  
(  
   $\exists (x: T) \Rightarrow S(x)$   
)
```

Proof(

```
1 (  $\exists (x: T) \Rightarrow (D(x) \wedge \neg B(x))$  )  
2 (  $\forall (x: T) \Rightarrow (\neg B(x) \rightarrow S(x))$  )
```

by Premise,

by Premise,

//COMPLETE THE PROOF HERE

```
3 Let ((a: T) => SubProof(  
  4 Assume  $(D(a) \wedge \neg B(a))$  ,  
  5 (  $\neg B(a) \rightarrow S(a)$  )  
  6 (  $\neg B(a)$  )  
  7 (  $S(a)$  )  
  8 (  $\exists (x: T) \Rightarrow S(x)$  )
```

by $\text{AllE}[T](2)$,

by $\text{AndE2}(4)$,

by $\text{ImplyE}(5, 6)$,

by $\text{ExistsI}[T](7)$

```
)),
```

```
9 (  $\exists (x: T) \Rightarrow S(x)$  )
```

by $\text{ExistsE}[T](1, 3)$

```
)
```

2. Use mathematical induction to prove that if n is a positive integer, then:

$$2^1 + 2^2 + \dots + 2^n = 2^{n+1} - 2$$

Base case. We must prove the claim for the smallest positive integer, $n = 1$. We have:

$$2^1 = 2$$

And:

$$2^{1+1} - 2 = 2$$

So the base case holds.

Inductive step. We assume the inductive hypothesis: that $2^1 + 2^2 + \dots + 2^k = 2^{k+1} - 2$ for some fixed positive integer k . We must prove that our claim holds for $k+1$. We have that:

$$\begin{aligned} 2^1 + 2^2 + \dots + 2^k + 2^{k+1} &= 2^{k+1} - 2 + 2^{k+1} \\ &= 2(2^{k+1}) - 2 \\ &= 2^{(k+1)+1} - 2 \end{aligned}$$

Thus the claim holds for $k+1$, and so the inductive step holds.

Thus by induction, $2^1 + 2^2 + \dots + 2^n = 2^{n+1} - 2$ for all natural numbers n .

3. Use a direct proof to prove that if x is an odd integer, then $6x+4$ is even.

Let x be an odd integer. Then there exists an integer k such that $x = 2k + 1$. We have that:

$$\begin{aligned} 6x + 4 &= 6(2k + 1) + 4 \\ &= 12k + 6 + 4 \\ &= 12k + 10 \\ &= 2(6k + 5) \end{aligned}$$

Thus $6x + 4 = 2b$, where b is the integer $6k + 5$. It follows from the definition of an even number that $6x + 4$ is even.

4. Consider the following questions about recursive definitions.

- a. Consider the pattern 3,8,13,18,23,... Give a recursive definition of the function $f(n)$, where $n \in \mathbb{N}$, that defines the pattern.

$$f(1) = 3$$

$$f(n) = f(n-1) + 5, \text{ if } n > 1$$

- b. Give a recursive definition of the function $SumOdds(n)$, where $SumOdds(n)$ is the sum of the first n odd integers.

$$SumOdds(1) = 1$$

$$SumOdds(n) = SumOdds(n-1) + (2n-1), \text{ if } n > 1$$

5. Consider the following program. Add appropriate proof blocks so that the assertion at the end would hold in Logika.

```
var x: Z = Z.prompt("Enter a positive number: ")
val input: Z = x

assume(x > 0)

x = x + 2

Deduce(
  1 ( input == Old(x) ) by Premise,
  2 ( x == Old(x) + 2 ) by Premise,
  3 ( x == input + 2 ) by Algebra*(1, 2),
  4 ( Old(x) > 0 ) by Premise,
  5 ( x > 2 ) by Algebra*(4, 2)
)

x = x * 2

Deduce (
  1 ( Old(x) > 2 ) by Premise,
  2 ( Old(x) == input + 2 ) by Premise,
  3 ( x == Old(x) * 2 ) by Premise,
  4 ( x == (input + 2) * 2 ) by Algebra*(2, 3),
  5 ( x == input*2 + 4 ) by Algebra*(4),
  6 ( x > 5 ) by Algebra*(1, 3),
  7 ( x == input*2 + 4 & x > 5 ) by AndI(5, 6)
)

//when you finish, the assertion below should hold

assert(x == input*2 + 4 & x > 5)
```

6. Consider the following partial program (the missing parts before and after the given loop are not relevant). Complete proof blocks **PART 1** and **PART 2** to finish the correctness proof for the loop invariants. You ONLY need to prove that the loop invariants still hold at the end of each iteration.

```
...
//assume x is some previously defined variable

var ans: Z = 2*x
var k: Z = 0

//assume loop invariants were already shown to hold before loop began

while (k < x) {
  Invariant(
    Modifies(k, ans),
    2*x == ans + k,
    k >= 0
  )

  ans = ans - 1

  Deduce(
    [PART 1]
    1 ( ans == Old(ans) - 1 ) by Premise,
    2 ( 2*x == Old(ans) + k ) by Premise,
    3 ( Old(ans) == ans + 1 ) by Algebra*(1),
    4 ( 2*x == ans + 1 + k ) by Algebra*(2, 3)
  )

  k = k + 1

  Deduce(
    [PART 2]

    1 ( Old(k) >= 0 ) by Premise,
    2 ( Old(k) < x ) by Premise,
    3 ( k == Old(k) + 1 ) by Premise,
    4 ( 2*x == ans + 1 + Old(k) ) by Premise,
    5 ( Old(k) == k - 1 ) by Algebra*(3),
    6 ( 2*x == ans + 1 + k - 1 ) by Algebra*(4, 5),
    7 ( 2*x == ans + k ) by Algebra*(6),
    8 ( k >= 0 ) by Algebra*(1, 3)
  )
}
...
```

7. Consider the following function, which computes and returns the absolute difference between two numbers. (Here, *absDiff*(7,3) returns 4 and *absDiff*(3,7) also returns 4, as $|7 - 3| = |3 - 7| = 4$, where $| \cdot |$ denotes absolute value.) **Add appropriate proof blocks** so that the function would be verified in Logika. To save space, you do not need to write “Deduce”.

```
def absDiff(a: Z, b: Z): Z = {
  Contract(
    Ensures(
      Res[Z] >= 0,
      Res[Z] == a-b V Res[Z] == b-a
    )
  )

  var diff: Z = 0
  var eval: Z = a-b

  if (eval >= 0) {
    diff = eval
    Deduce(
      1 ( eval >= 0 ) by Premise,
      2 ( eval == a - b ) by Premise,
      3 ( diff == eval ) by Premise,
      4 ( diff >= 0 ) by Algebra*(1, 3),
      5 ( diff == a - b ) by Algebra*(2, 3)
    )
  } else {
    diff = -1*eval
    Deduce(
      1 ( !(eval >= 0) ) by Premise,
      2 ( eval == a-b ) by Premise,
      3 ( eval < 0 ) by Algebra*(1),
      4 ( diff == -1*eval ) by Premise,
      5 ( diff >= 0 ) by Algebra*(3, 4),
      6 ( diff == b-a ) by Algebra*(2, 4)
    )
  }

  Deduce(
    1 ( diff >= 0 ) by Premise,
    2 ( diff == a - b V diff == b - a ) by Premise
  )

  return diff
}
```

8. Consider the following shell of a program. Here, much of the code and reasoning is missing. Assume that *sampleFn* correctly verifies its function contract. **Add appropriate proof blocks under “Calling code”** so that the assert at the end would hold in Logika. (Again, assuming that the function was correctly verified.)

```
def sampleFn(x:Z, y:Z): Z = {
  Contract(
    Requires( x+y > 0 ),
    Ensures( Res[Z] == 2*x - y )
  )

  ...(code/verification for function)
}

////////// Calling code //////////

var a: Z = 4
var b: Z = 2

Deduce(
  1 ( a == 4 ) by Premise,
  2 ( b == 2 ) by Premise
  3 ( a-1 + b > 0 ) by Algebra*(1,2)
)

var back: Z = sampleFn(a-1, b)

Deduce(
  1 ( back == 2*(a-1) - b ) by Premise,
  2 ( a == 4 ) by Premise,
  3 ( b == 2 ) by Premise,
  4 ( back == 2*3 - 2 ) by Algebra*(1,2,3),
  5 ( back == 4 ) by Algebra*(4)
)

assert(back == 4)
```